Does a meta-learned autoencoder actually adapt? A measurement study of first-order MAML for few-shot anomaly detection on industrial and spacecraft telemetry

Abhijeet Chandrakant Karve[1] (corresponding author, ackarve11@gmail.com)

Santosh Darade[1]

[1]Department of Computer Science and Engineering, MIT Art, Design and Technology University, Pune, India

Abstract

Meta-learning is regularly proposed as the way to give an industrial anomaly detector a few-shot start in a new operating regime, and the evidence offered is usually a score table for the adapted model. That table cannot show whether the meta-learner trained, whether the K-shot adaptation changed the model, or how much of the score the model itself contributed. We ran a measurement study that answers those three questions for a plain detector, an LSTM autoencoder trained with first-order MAML on regimes carved from normal operation, on five public benchmarks (the SWaT, WADI and HAI 21.03 testbeds and the SMAP and MSL telemetry sets), The study uses five seeds, support sizes from 1 to 100 windows, within-plant and cross-plant evaluation, strictly point-wise metrics and a threshold set without labels. Every comparison is a paired test on the same (seed, stream, K) cell, 11,180 result rows in all. Four instruments were added to the usual protocol: the norm of the meta-update per outer step, with a control that reproduces a common gradient-path fault in first-order MAML code that silently leaves the meta-model at its initialisation; the relative parameter change and the rank correlation of anomaly scores before and after adaptation, at three adaptation strengths; a parameter-free reference, the squared magnitude of the standardised window; and a per-stream task-diversity score. The corrected meta-learner trains, and the broken one leaves a signature only in the update norm, which was exactly zero in all 25 control runs while its loss curve looked ordinary. K-shot adaptation moved the autoencoder by 0.1–0.6 % of its parameter norm at the settings in use and left the ranking of windows with a rank correlation of 0.93–0.99; it changed AUROC by a few thousandths everywhere except SMAP, where it repaired a poor meta-initialisation to the level of a static model. The meta-learned detector never beat the static autoencoder by more than 0.01 AUROC within a plant, was 0.056 worse on HAI, and matched a statically pre-trained source model in all six transfer directions. On four of the five benchmarks no autoencoder, trained or meta-trained, beat the window magnitude, which a random untrained network reproduces with rank correlation 0.996–1.000; only HAI rewarded training, and there a flat autoencoder beat the recurrent one. The label-free threshold cost 0.06–0.25 F1 against the oracle, on four of the five benchmarks more than any modelling choice. Task diversity did not predict the meta-learning gain across 120 telemetry channels (Spearman −0.20). We release the code and tables and propose five measurements that a report of a meta-learned detector should include, so that a reader can see whether the adaptation did anything.

Keywords: meta-learning; MAML; LSTM autoencoder; anomaly detection; industrial control systems; spacecraft telemetry; few-shot adaptation; reproducibility.

1 Introduction

A reconstruction-based anomaly detector for a plant is trained on normal sensor data and raises an alarm when a window reconstructs badly. The appeal of meta-learning in this setting is easy to state. A plant has many operating regimes, a new plant or a new line has few recordings, and a model that can be pulled into a

new regime with a handful of normal windows would save the weeks of data collection that a static detector needs. A number of papers report that Model-Agnostic Meta-Learning (MAML) [1] or one of its variants delivers this for industrial fault detection [2], [3], [4], [5]. Most of them report the detection score of the adapted model and little else.

That reporting habit hides two questions that a reader cannot answer from a score table. The first is whether meta-training happened at all. The inner loop of MAML works on a copy of the model, and in a framework such as PyTorch it is easy to back-propagate the query loss on the copy and then step the optimiser on the original. The original then holds no gradient, the step does nothing, and the training curve still goes down because the inner loop keeps lowering the support loss on each fresh copy. The code runs, nothing is printed, and the "meta-learned" model that is evaluated is the random initialisation. We found exactly this pattern in our own earlier implementation, and the empirical studies of deep-learning faults [6], [7] list this class of silent, non-crashing training bug among the most common. The second question is whether the K-shot adaptation that the method is named after changes the detector in any measurable way. Ten gradient steps on twenty windows with a small learning rate may move an LSTM autoencoder with a hundred thousand parameters so little that its anomaly ranking is unchanged, in which case the reported score belongs to the meta-initialisation and the adaptation is decoration. Raghu et al. [8] showed for image classification that MAML's benefit comes mostly from feature reuse in the initialisation rather than from the inner loop; whether the same holds for a sequence autoencoder on sensor data has, to our knowledge, not been measured.

This paper is a measurement study built around those two questions. We keep the detector deliberately plain, an LSTM autoencoder [9] trained with first-order MAML [10], so that nothing about the architecture distracts from what is being measured. We then add the instruments that the usual report leaves out. The first is the norm of the meta-parameter update at every outer step, with the broken gradient path run next to the corrected one as a control. The second is the relative parameter change and the rank correlation of anomaly scores before and after K-shot adaptation, at three adaptation strengths. The third is a deployed threshold set without labels, printed next to the oracle best F1 so that the cost of the threshold rule is visible. The fourth is a task-diversity score per stream, which lets us test rather than assert whether meta-learning gains follow the spread of operating regimes. The study runs on five public benchmarks, the SWaT, WADI and HAI 21.03 testbeds and the SMAP and MSL spacecraft telemetry sets, with five seeds, four support sizes from one to a hundred windows, within-plant evaluation on held-out channels or attack streams, and cross-plant transfer between the three testbeds through a shared principal-component space. Every comparison is paired on the same seed, entity and support size and tested with a Wilcoxon signed-rank test.

The questions we answer are these. (Q1) Does the corrected first-order MAML train, and what does the broken pattern produce instead? (Q2) How far does K-shot adaptation move the autoencoder, and does it change which windows are ranked as anomalous? (Q3) Once (Q1) and (Q2) are settled, does meta-training or adaptation improve detection over a static autoencoder, a flat autoencoder and Isolation Forest, within a plant and across plants, and does any gain follow task diversity? The contributions are the instrumented protocol, the control experiment for the gradient-path fault, the measurements themselves, and a short list of what a paper that reports a meta-learned detector should show.

The rest of the paper is organised as follows. Section 2 sets out what is claimed when a meta-learned detector is reported and what the usual evidence leaves open. Section 3 describes the detector, the meta-learner, the control and the measurements. Section 4 gives the benchmarks and the protocol. Section 5 reports the results in the order of the three questions. Section 6 discusses what they mean for practice and lists the limits of the study, and Section 7 concludes.

2 Background: what a meta-learned detector claims

2.1 Reconstruction-based detection on sensor windows

A detector sees a multivariate stream $x_t \in \mathbb{R}^d$ in windows $w_t = (x_{t-L+1}, \ldots, x_t)$ of length L. An autoencoder trained on normal windows maps a window to a code and back, and the squared reconstruction error $e(w) = \| w - \hat{w} \|^2 / (Ld)$ is the anomaly score. Sakurada and Yairi [11] used a flat autoencoder for spacecraft telemetry; Malhotra et al. [9] replaced it with an LSTM encoder-decoder so that the code summarises the order of readings as well as their values, and that design, with later variants [12], [13], [14], is the standard reconstruction detector for industrial streams [15], [16], [17]. A score becomes an alarm through a threshold, and the threshold is where most of the practical difficulty sits: the labels needed to pick it are exactly what the setting lacks, and the "best F1 over all thresholds" that many papers report is an oracle quantity [18], [19], [20]. We set the threshold with the peaks-over-threshold rule of Siffer et al. [21] on a label-free calibration stretch and report the oracle number beside it.

2.2 Optimisation-based meta-learning

MAML [1] looks for an initialisation θ from which a few gradient steps on the support set S_i of a task give a model that does well on that task's query set Q_i. The inner step is $\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{S_i}(\theta)$, repeated n times, and the outer step moves θ along $\nabla_\theta \sum_i \mathcal{L}_{Q_i}(\theta_i')$. The first-order variant [10] drops the Hessian term and uses $\nabla_{\theta_i'} \mathcal{L}_{Q_i}(\theta_i')$ in its place, which is what makes it cheap enough for a recurrent autoencoder. MAML++ [22] and Meta-SGD [23] learn the inner learning rates; Hospedales et al. [24] survey the family and Wang et al. [25] the wider few-shot literature. For an unsupervised detector a "task" is a slice of normal operation, and the support and query sets are normal windows from that slice; the model is never shown an anomaly during meta-training.

The first-order variant has an implementation hazard that the second-order one does not. The inner loop must run on a copy of θ (a functional call or a deep copy), and the query gradient is then a gradient with respect to the copy's parameters. It has to be transferred onto θ before the outer optimiser steps. If the query loss is instead back-propagated on the copy and the optimiser is stepped on the original, the original's gradient buffers are empty, the optimiser does nothing, and θ stays at its initial value for the whole run. Nothing fails: the support loss inside each episode still falls because the copy is being trained, and a logged "meta-training loss" that averages that support loss looks like learning. We call this the gradient-path fault. The repair is one line, and Section 3.2 gives it. What matters for this paper is not the fix. It is that the fault cannot be seen in the quantities most papers report, so a control that reproduces it is the only way a reader can be sure a result came from a trained meta-model.

2.3 What the evidence usually shows, and what it leaves open

The industrial meta-learning papers we are aware of [2], [3], [4], [5] report the score of the adapted model against baselines, sometimes against a non-meta-learned version of the same network. None reports the size of the parameter change that adaptation produced, the correlation between anomaly scores before and after adaptation, a control for the training itself, or a label-free threshold. Raghu et al. [8] is the closest methodological precedent: on few-shot image classification they froze the body of a MAML-trained network during adaptation and lost almost nothing, so the inner loop was doing little beyond re-fitting the head. An autoencoder has no head, so the analogous question is whether the inner loop moves the whole network at all, and by how much. Henderson et al. [26] and Pineau et al. [27] make the general case that seeds, paired comparisons and released code are the minimum for a claim of improvement; we follow that.

Two further gaps motivate the design. First, benchmark evaluation of time-series detectors has well-documented traps [18], [19], [20], the largest being point-adjusted scoring, which credits a whole anomaly segment for one hit, and oracle thresholds. We score strictly point-wise. Second, the explanation most often

offered for a weak meta-learning result, that the tasks were not diverse enough, is rarely measured. We measure it with the silhouette [28] of a clustering of normal windows into regimes, per stream, and correlate it with the gain over the static model across all held-out streams, which gives the explanation a sample size rather than an anecdote.

This paper is deliberately an offline study: the meta-model is trained once, adapted once with K windows, and scored on a fixed test stream. Online adaptation on a drifting stream, where a drift detector decides when to adapt, is a different problem with its own literature [29], [30], and we keep it out of this study. Keeping the two apart matters here: the measurements are about the meta-learner itself, with no gate, memory or threshold logic in the way.

3 Detector, meta-learner, control and instruments

Figure 1 shows the whole study as a pipeline: how tasks are built from one plant’s normal data, the detector, the meta-training loop with the broken-pattern control beside the corrected one, what happens at test time, and the cross-plant bridge. The subsections that follow give the settings for each band.

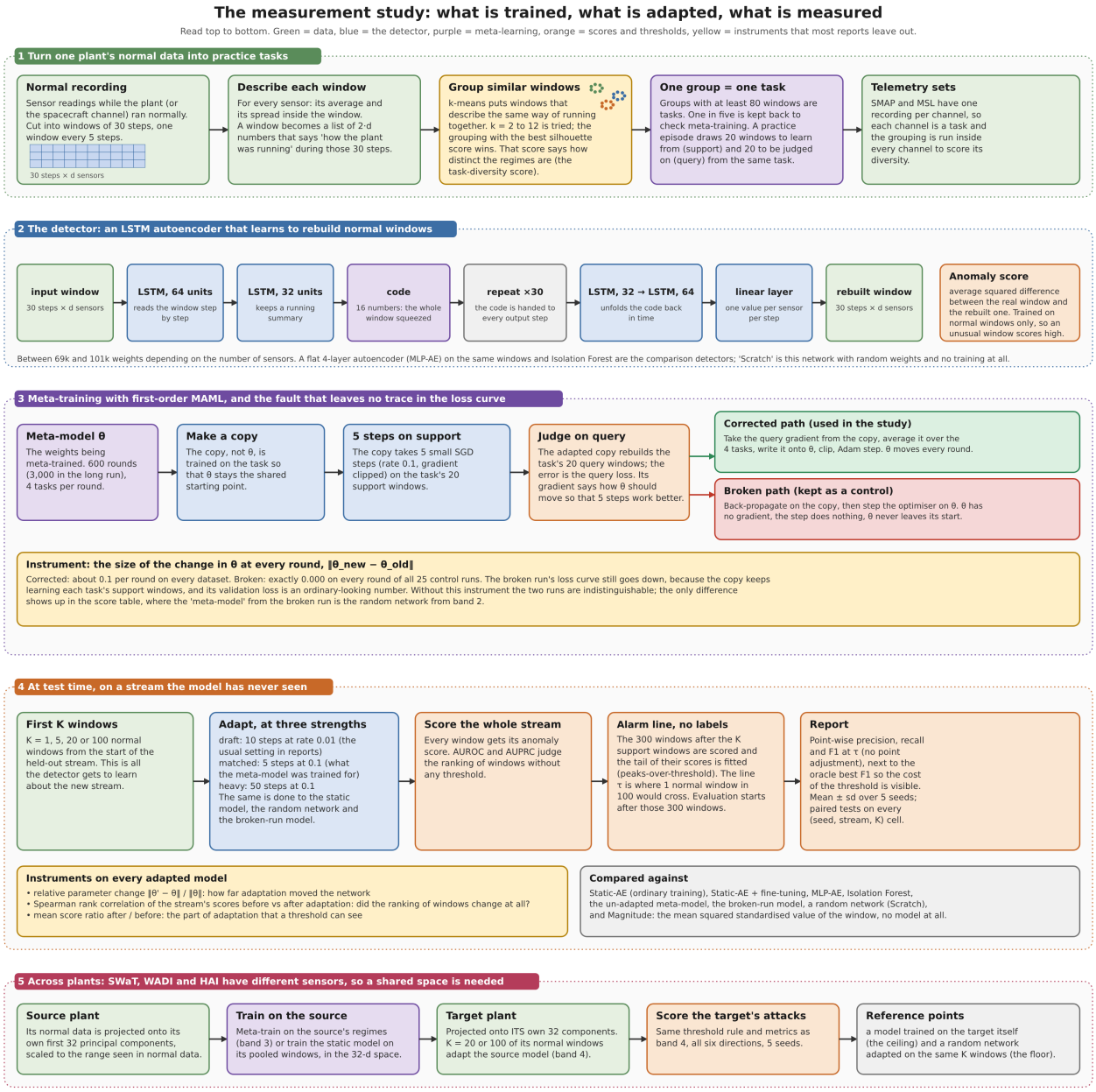


Figure 1. The study as a pipeline. Band 1: normal windows are described by their per-sensor mean and spread and grouped into operating regimes, which are the meta-learning tasks; the silhouette of the grouping is the task-diversity score. Band 2: the LSTM autoencoder and its anomaly score. Band 3: one round of first-order MAML, with the corrected gradient path and the broken one kept as a control, and the per-round parameter-change instrument that tells them apart. Band 4: K-shot adaptation at three strengths, label-free thresholding, point-wise scoring, the three adaptation instruments and the reference detectors. Band 5: the principal-component bridge for cross-plant transfer.

3.1 The LSTM autoencoder

The detector is the encoder-decoder of Malhotra et al. [9] with two LSTM layers [31] on each side. The encoder reads a $30 \times d$ window through LSTM layers of 64 and 32 units and maps the final hidden state to a 16-dimensional code. The decoder repeats the code over 30 steps, passes it through LSTM layers of 32 and 64 units and a linear output layer, and the loss is the mean squared error over the window. The model has between 72k and 101k parameters depending on d. The anomaly score of a window is its mean squared reconstruction error. A flat autoencoder on the same windows (a four-layer multilayer perceptron with 128 hidden units and the same 16-dimensional code, ELU activations) is the “does the recurrence matter” control, and Isolation Forest [32] on flattened windows is the non-neural reference. The static version of each

network is trained with Adam [33] at learning rate 10^{-3} for 2,000 mini-batches of 64 windows with early stopping on a 10 % held-out slice of the training windows.

3.2 First-order MAML and the gradient-path control

Meta-training draws a batch of four tasks per outer step. For each task the support set S (20 windows) adapts a deep copy of the meta-model for five SGD steps at inner rate $\alpha = 0.1$ with the gradient norm clipped at 1.0, and the query set Q (20 further windows from the same task) is scored by the adapted copy. The corrected update takes the gradient of the query loss with respect to the copy's parameters, averages it over the four tasks, writes it into the gradient buffers of the meta-model, clips it and steps Adam at 10^{-3}. We run 600 outer steps and keep the meta-parameters with the lowest post-adaptation query loss on a fixed validation task built from regimes that are held out of meta-training, evaluated every 100 steps.

The control runs the same loop with the fault left in: the query loss is back-propagated on the copy and the optimiser is stepped on the original. We log, at every outer step of both runs, the Euclidean norm of the change in the meta-parameters, $\| \theta_{k+1} - \theta_k \|$, and the validation query loss. The broken run is limited to 100 outer steps because its outcome is decided by the first; it is kept in every seed and dataset so that its signature is reported, not remembered. The models adapted and scored at evaluation time from the broken meta-model are labelled "MAML-AE-broken" and are, by construction, adaptations of the random initialisation; a second random initialisation with a different seed, labelled "Scratch", shows how much of their behaviour is the initialisation and how much is the adaptation.

Inner learning rate matters for the question in (Q2), so it was set by a short pilot rather than copied from the classification literature. With $\alpha = 0.01$, the value in our earlier draft and in many reports, five inner steps moved the autoencoder by less than 0.2 % of its parameter norm and the support loss by a few per cent; $\alpha = 0.1$ moved it by about 2 % and roughly halved the support loss; meta-training was stable at both and at 0.3. We train at 0.1 and evaluate adaptation at three settings described next.

3.3 Tasks from one plant

The three testbeds each provide one long normal recording (HAI provides three, recorded weeks apart, which we keep as separate streams). Tasks are carved from it by clustering. Every normal window is summarised by its per-sensor mean and standard deviation, a $2d$ vector; k-means is run for k from 2 to 12 on up to 5,000 windows, the k with the highest silhouette [28] is kept, and each cluster with at least 80 windows is a task. One cluster in five, at least one, is held out as the meta-validation task. The silhouette of the chosen clustering is recorded as the task-diversity score of the stream. For the two telemetry sets, where each channel has its own normal recording, the channels of the meta-training split are the tasks and the same clustering is run inside every channel to give a per-channel diversity score; it is this per-channel score, over 60 SMAP and MSL channels, that the diversity analysis in Section 5.6 uses.

3.4 Adaptation settings and the adaptation instruments

At evaluation a model is adapted on the first K windows of the held-out test stream, $K \in \{1, 5, 20, 100\}$, which are normal in every stream used. Three adaptation strengths are scored. *Draft* is ten SGD steps at 0.01, the setting of our earlier implementation and of the typical report. *Matched* is five steps at 0.1, the inner loop the meta-model was trained for. *Heavy* is fifty steps at 0.1, a deliberate over-adaptation. All three clip the gradient norm at 1.0. The same procedure is applied to the static autoencoder ("Static-AE+FT"), so that the comparison between meta-initialisation and ordinary initialisation is made with the same adaptation.

For every adapted model we record three numbers. The *relative parameter change* $\| \theta' - \theta \| / \| \theta \|$ says how far adaptation moved the network. The *score rank correlation* is Spearman's ρ between the anomaly

scores of the evaluation part of the stream before and after adaptation; a ρ of 1.0 means adaptation did not change which windows look anomalous, whatever it did to the scale. The *score mean ratio* is the mean score after adaptation divided by the mean before, which is the part of adaptation that a threshold can see. Together they separate three outcomes that a score table conflates: adaptation that does nothing, adaptation that rescales the errors without reordering them, and adaptation that changes the ranking.

3.5 Threshold and scoring

The deployed threshold is set once per (stream, K) without labels: the 300 windows after the K support windows (200 for the shorter telemetry streams) are scored, and a generalised-Pareto tail is fitted to their upper 2 % to give the level exceeded with probability 10^{-2} [21]. Evaluation starts after the calibration stretch. Precision, recall and F1 are strictly point-wise at that threshold, with no point adjustment. AUROC and AUPRC summarise the ranking independently of the threshold, and the best F1 over all thresholds is reported as the oracle quantity it is, so that the gap between it and the deployed F1 is visible. Methods that do not adapt are scored once per stream, and their threshold is recalibrated for each K on the same stretch, so that every method at a given K sees the same evaluation windows.

4 Benchmarks and protocol

4.1 Datasets

Two of the benchmarks are the water testbeds of iTrust at the Singapore University of Technology and Design, which we use under its data-use agreement: *SWaT* [34], a six-stage treatment plant with 51 sensors and actuators and 36 labelled attacks, and *WADI* [35], a distribution network with 123 usable tags (four channels that never move are removed) and 15 attacks. We take every tenth second of both and discard the first six hours of SWaT's normal recording, during which the plant is still settling. The third testbed is *HAI 21.03* [36], [37], where a boiler, a turbine and a water loop are driven by a hardware-in-the-loop simulator; its 79 tags come as three attack-free recordings and five recordings with attacks, which we average over ten-second blocks and evaluate one by one. The two telemetry sets are the *SMAP* and *MSL* channels released with telemanom [38]: 55 channels of 25 features from the Soil Moisture Active Passive satellite and 27 channels of 55 features from the Mars Science Laboratory rover, each channel with a normal recording and a labelled test recording. For the two telemetry sets 30 % of channels are held out per seed and the rest are meta-training tasks; for the testbeds the normal recording supplies the tasks and every attack recording is evaluated. Values are standardised with the stream's own normal statistics and clipped at ±10. Table 1 summarises the streams.

Table 1. Benchmarks. Test length is in windows; the anomaly rate is the fraction of labelled test steps. Held-out counts are per seed.

Dataset	Domain	d	Streams (held out)	Test length	Anomaly rate
SWaT	water treatment testbed	51	1 (1)	45k	12 %
WADI	water distribution testbed	123	1 (1)	17k	5.8 %
HAI 21.03	boiler/turbine/water testbed	79	5 (5)	4k–12k	1.4–2.9 %
SMAP	spacecraft telemetry	25	55 (17)	4.4k–8.6k	12 % avg.
MSL	rover telemetry	55	27 (8)	1.1k–6.1k	12 % avg.

4.2 Within-plant protocol

For each dataset and seed the meta-training tasks are built as in Section 3.3. The static autoencoder, the flat autoencoder and Isolation Forest are trained on the pooled normal windows (a stride of 5 between windows for the testbeds and 2 for telemetry, capped at 20,000 windows). The corrected meta-model and the broken control start from the same initialisation. Every held-out stream is then scored at every K by every method and adaptation setting. The methods are: Isolation Forest; MLP-AE; Static-AE; Static-AE+FT at the three settings; MAML-AE-noadapt (the meta-initialisation scored without adaptation); MAML-AE at the three settings; MAML-AE-broken and Scratch at the matched and heavy settings, with their un-adapted versions. Five seeds give five meta-training runs, five static models and five held-out splits per dataset. The unit of analysis is a cell (dataset, seed, stream, K); a dataset's number for a method is the mean over its streams within a seed and then the mean and standard deviation over seeds, so that a dataset with 17 held-out channels does not outvote one with a single stream.

4.3 Cross-plant protocol

The three testbeds have different sensors, so a model trained on one cannot read another. We bridge them with principal components [39]: each plant's normal recording is projected onto its own first 32 components, scaled to the unit range of its normal data and clipped, and the autoencoder works in that 32-dimensional space. The meta-model is trained on the source plant's regimes, the static source model on its pooled windows, and both are adapted on $K \in \{20, 100\}$ target windows at the matched and heavy settings and scored on the target's attack recording(s). "Target-static" is an autoencoder trained on the target plant's own normal recording in the same projected space, which is the ceiling a transfer method is trying to reach; "Source-static-noadapt" and "MAML-transfer-noadapt" show what the two source models do before adaptation, and "Scratch" shows what K windows alone do to a random network. All six directions among SWaT, WADI and HAI are run with five seeds.

4.4 Statistics and implementation

Paired differences are formed on cells and tested with the two-sided Wilcoxon signed-rank test; we report the mean difference, the win/loss count and the p-value and treat $p < 0.05$ as a difference worth discussing. Everything runs on CPU with PyTorch, one thread per job; a within-plant job takes 10–40 minutes depending on the number of held-out streams, and the whole study is 25 within-plant and 30 transfer jobs. The loaders, the meta-learner, the control, the runner and the table scripts are released with the paper.

5 Results

The study produced 9,920 within-plant rows (5 datasets, 5 seeds, 77 distinct held-out streams, 4 support sizes, 11 method/setting combinations plus the magnitude reference) and 1,260 transfer rows. Tables report the mean and standard deviation over seeds after averaging over a dataset's streams within each seed; paired tests use the cells. A difference in AUROC below 0.01 is treated as negligible whatever its p-value, because with 100 or more cells the Wilcoxon test flags differences of 0.0003.

5.1 Q1: the meta-learner trains, and the broken pattern leaves no trace in the loss

Table 2 gives the control for all 25 (dataset, seed) runs. In every one of them the broken pattern produced a parameter change of exactly zero at every outer step, the meta-parameters at the end were bit-for-bit the initialisation, and the validation query loss after adaptation was the same number at step 50 and step 100. The corrected loop moved the parameters by 0.08–0.11 in Euclidean norm per step and brought the validation

query loss down between step 100 and step 600, from 0.56 to 0.41 on SWaT, 0.81 to 0.68 on WADI and 0.77 to 0.66 on HAI, with smaller movements on the two telemetry sets whose losses start low (Figure 2).

Table 2. Gradient-path control, mean over five seeds. $\|\Delta\theta\|$ is the Euclidean norm of the meta-parameter change per outer step; "val" is the post-adaptation query loss on the held-out validation regime at outer step 100 and at step 600. The broken run is 100 steps; its validation loss at step 50 and at step 100 is the same number in all 25 runs, and its parameters at the end differ from the initialisation by exactly zero in all 25.

Dataset	corrected $\|\Delta\theta\|$ per step	corrected val, step 100 → 600	broken $\|\Delta\theta\|$ per step	broken max	$\theta - \theta_0$	
SWaT	0.094	0.555 → 0.406	0.000	0.000	0.773, 0.773	0.129
WADI	0.107	0.809 → 0.675	0.000	0.000	0.829, 0.829	0.221
HAI	0.100	0.767 → 0.659	0.000	0.000	0.704, 0.704	0.291
SMAP	0.102	0.378 → 0.335	0.000	0.000	0.421, 0.421	0.189
MSL	0.078	0.133 → 0.129	0.000	0.000	0.146, 0.146	0.112

Two things in the table matter for what follows. First, the only quantity that separates a working meta-learner from a broken one is the parameter change; the broken run's validation loss is not absurd (0.70 on HAI against 0.66 for the corrected run at step 600, 0.146 against 0.129 on MSL), so a loss curve alone would not have raised an alarm. Second, even the corrected meta-model finishes with a post-adaptation query loss between 1.2 (MSL) and 3.1 (SWaT, WADI) times the validation loss of the static autoencoder trained on the same windows. The two numbers are not on the same data (the meta-model is scored on a regime it never saw, the static model on a random slice of the pooled windows), but the gap is large enough to say that 600 outer steps of four tasks do not bring the meta-model to the reconstruction quality of 2,000 ordinary mini-batches. Section 5.7 reports what a five-fold longer meta-training changes.

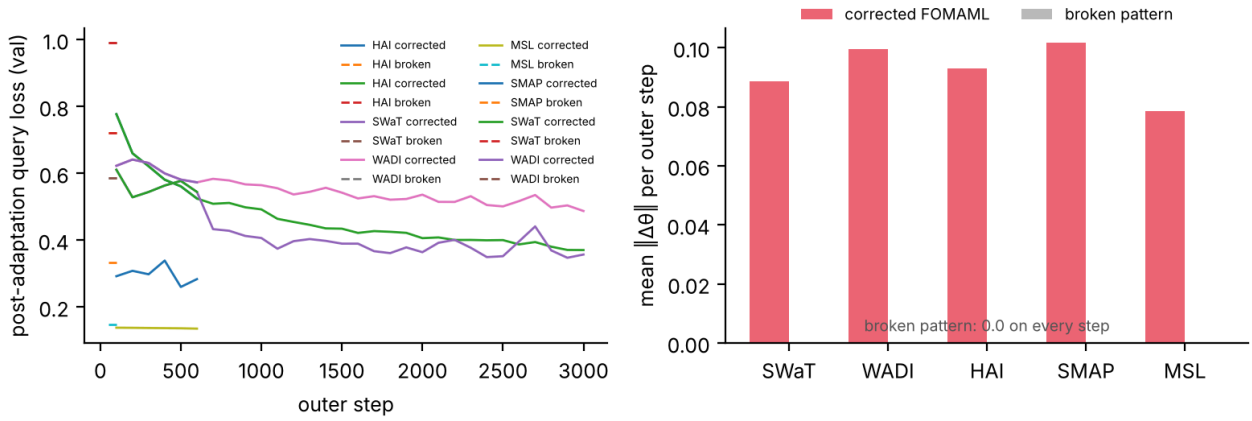


Figure 2. Left: post-adaptation validation query loss of the corrected meta-learner (solid) and of the broken pattern (dashed, flat by construction) over outer steps, seed 0. Right: mean parameter change per outer step; the broken pattern's bar is zero on every dataset.

5.2 Q3, within a plant: the meta-learned model does not beat the static one, and neither beats the magnitude of the window on three benchmarks

Tables 3 and 4 give AUROC and AUPRC at K = 20 for every method; Figure 3 shows AUROC against K. The first row is the detector with no parameters: the mean squared standardised value of the window.

Table 3. Within-plant detection at K = 20, AUROC, mean ± sd over five seeds. Scratch is a random LSTM autoencoder from a seed the meta-learner did not use.

method	SWaT	WADI	HAI	SMAP	MSL
Magnitude	0.859 ± 0.000	0.673 ± 0.000	0.855 ± 0.000	0.605 ± 0.031	0.610 ± 0.054
Isolation Forest	0.832 ± 0.001	0.718 ± 0.007	0.633 ± 0.006	0.559 ± 0.030	0.574 ± 0.055
MLP-AE	0.862 ± 0.004	0.579 ± 0.008	0.933 ± 0.001	0.599 ± 0.060	0.573 ± 0.059
Static-AE	0.867 ± 0.007	0.584 ± 0.004	0.901 ± 0.004	0.587 ± 0.052	0.592 ± 0.067
Static-AE+FT (matched)	0.867 ± 0.008	0.582 ± 0.007	0.900 ± 0.004	0.592 ± 0.046	0.585 ± 0.061
MAML-AE, no adaptation	0.858 ± 0.002	0.589 ± 0.042	0.845 ± 0.021	0.567 ± 0.043	0.588 ± 0.058
MAML-AE (matched)	0.859 ± 0.003	0.592 ± 0.041	0.846 ± 0.023	0.597 ± 0.043	0.589 ± 0.059
MAML-AE (heavy)	0.856 ± 0.003	0.586 ± 0.047	0.844 ± 0.031	0.594 ± 0.044	0.591 ± 0.057
MAML-AE-broken (matched)	0.858 ± 0.000	0.672 ± 0.001	0.854 ± 0.001	0.605 ± 0.033	0.606 ± 0.055
Scratch (matched)	0.859 ± 0.000	0.672 ± 0.001	0.854 ± 0.001	0.607 ± 0.033	0.609 ± 0.058

Table 4. Within-plant detection at K = 20, AUPRC, mean ± sd over five seeds.

method	SWaT	WADI	HAI	SMAP	MSL
Magnitude	0.769 ± 0.000	0.181 ± 0.000	0.280 ± 0.000	0.245 ± 0.046	0.307 ± 0.054
Isolation Forest	0.726 ± 0.005	0.200 ± 0.017	0.053 ± 0.004	0.181 ± 0.050	0.244 ± 0.032
MLP-AE	0.776 ± 0.002	0.100 ± 0.014	0.344 ± 0.001	0.245 ± 0.065	0.241 ± 0.071
Static-AE	0.769 ± 0.008	0.101 ± 0.011	0.321 ± 0.008	0.241 ± 0.059	0.287 ± 0.059
Static-AE+FT (matched)	0.769 ± 0.009	0.101 ± 0.013	0.319 ± 0.008	0.250 ± 0.064	0.288 ± 0.058
MAML-AE, no adaptation	0.764 ± 0.003	0.114 ± 0.026	0.288 ± 0.004	0.226 ± 0.050	0.288 ± 0.059
MAML-AE (matched)	0.764 ± 0.004	0.116 ± 0.024	0.284 ± 0.006	0.237 ± 0.054	0.288 ± 0.060
MAML-AE (heavy)	0.762 ± 0.005	0.116 ± 0.022	0.283 ± 0.015	0.255 ± 0.064	0.288 ± 0.059
MAML-AE-broken (matched)	0.769 ± 0.000	0.180 ± 0.001	0.280 ± 0.001	0.241 ± 0.050	0.306 ± 0.054
Scratch (matched)	0.769 ± 0.000	0.182 ± 0.001	0.279 ± 0.001	0.242 ± 0.049	0.308 ± 0.054

Three rows of the table agree with each other. The magnitude reference, the random network and the “meta-model” from the broken loop agree to three decimals on SWaT, WADI and HAI and to within 0.005 on the telemetry sets. The reason is mechanical: an untrained LSTM autoencoder emits an output of small, nearly constant amplitude, so its reconstruction error is the squared norm of the input up to an offset. On the first 6,000 test windows of each dataset the Spearman correlation between the error of a random network and the window magnitude is 0.996 to 1.000. A paper that evaluated the broken loop would have reported the magnitude detector under the name of a meta-learned one. On SWaT that would have been a respectable 0.858.

Against that reference the learned detectors do the following. On SWaT the static autoencoder gains 0.009 AUROC over magnitude (16 of 20 cells, $p = 0.0001$) and the meta-learned one gains nothing (0.0001, $p = 0.93$); the static model beats the meta-learned one by 0.009 (17 of 20 cells, $p < 0.0001$). On HAI the flat

MLP autoencoder is the best detector at 0.933, the static LSTM autoencoder follows at 0.901, and the meta-learned LSTM autoencoder is at 0.846, below the magnitude reference by 0.009 (p = 0.025) and below the static model by 0.056 in 89 of 100 cells. On WADI training hurts: every trained autoencoder is between 0.58 and 0.59, the magnitude reference is at 0.673 and Isolation Forest at 0.718; the static model loses 0.089 to magnitude in all 20 cells. On SMAP and MSL no method is far from 0.6; the meta-learned and static models differ by 0.005 (p = 0.29) and 0.003 (p = 0.0003), both negligible, and both sit 0.014–0.021 below the magnitude reference. The meta-learned model never beats the static model by more than 0.01 on any dataset, and the one dataset where the sign favours it (WADI, +0.008) has p = 0.33.

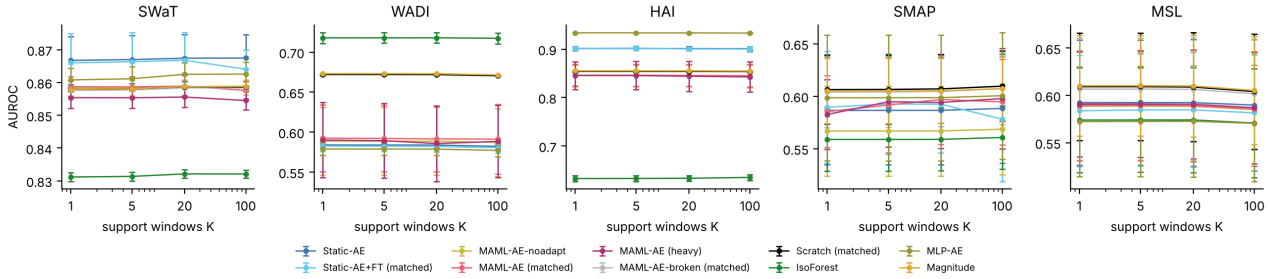


Figure 3. AUROC against the number of support windows K, mean ± sd over seeds. The curves are flat: K changes AUROC by at most 0.01 for any method on any dataset.

The support size does nothing. Between K = 1 and K = 100 the AUROC of every method on every dataset changes by less than 0.012, and the only change above 0.005 is the meta-learned model on SMAP (0.585 to 0.595). This is expected for the methods that do not adapt, whose only dependence on K is the position of the calibration stretch, and it is the first sign that the adapting ones barely adapt.

5.3 Q2: K-shot adaptation moves the autoencoder very little and reorders almost nothing

Table 5 reports the three adaptation instruments pooled over datasets (per-dataset values are in Figure 4 and the released tables). At the draft setting, ten steps at 0.01, adaptation changes 0.2 % of the meta-model's parameter norm and 0.1 % of the static model's, and the rank correlation between scores before and after is 0.95–0.97 for the meta-model and 0.99 for the static one. The matched setting, the inner loop the meta-model was trained for, moves it by 0.5 % and reorders slightly more (ρ = 0.93–0.95). The heavy setting, fifty steps at 0.1, moves it by about 2 % and brings ρ down to 0.87–0.89, which is the only setting under which the evaluation ranking of windows changes in a way a reader would notice; on HAI the heavy setting takes ρ to 0.76 ± 0.13.

Table 5. Adaptation instruments pooled over the five datasets (mean ± sd over seeds of dataset means). ΔAUROC is relative to the same model before adaptation on the same cell.

model / setting	K	rel. parameter change	score rank ρ	mean score ratio	ΔAUROC
MAML-AE / draft	1	0.002 ± 0.000	0.973 ± 0.024	0.953 ± 0.039	+0.009 ± 0.013
MAML-AE / draft	20	0.002 ± 0.000	0.958 ± 0.027	0.937 ± 0.045	+0.015 ± 0.012
MAML-AE / matched	1	0.005 ± 0.001	0.952 ± 0.029	0.928 ± 0.042	+0.010 ± 0.009
MAML-AE / matched	20	0.005 ± 0.001	0.940 ± 0.037	0.917 ± 0.045	+0.016 ± 0.010
MAML-AE / matched	100	0.006 ± 0.001	0.929 ± 0.050	0.876 ± 0.067	+0.014 ± 0.007
MAML-AE / heavy	20	0.017 ± 0.002	0.887 ± 0.059	0.917 ± 0.040	+0.014 ± 0.004

model / setting	K	rel. parameter change	score rank ρ	mean score ratio	ΔAUROC
MAML-AE / heavy	100	0.020 ± 0.002	0.871 ± 0.073	0.871 ± 0.068	+0.015 ± 0.008
Static-AE+FT / draft	20	0.001 ± 0.000	0.993 ± 0.006	0.992 ± 0.005	−0.001 ± 0.003
Static-AE+FT / matched	20	0.003 ± 0.000	0.952 ± 0.030	1.173 ± 0.085	+0.001 ± 0.005
Static-AE+FT / heavy	1	0.012 ± 0.001	0.890 ± 0.046	1.219 ± 0.102	+0.004 ± 0.005
Static-AE+FT / heavy	100	0.015 ± 0.001	0.934 ± 0.037	1.164 ± 0.044	−0.002 ± 0.006

What adaptation does change is the scale of the scores. For the meta-model the mean score after adaptation is 0.88–0.95 of the mean before, that is, adaptation lowers reconstruction error across the whole stream; for the static model the heavy setting raises the mean score by 16–22 %, because fine-tuning a converged network on one to a hundred windows pulls it towards those windows and away from the rest. Neither change is visible in AUROC, which is scale-free, but both are visible to a threshold, which is why the instruments have to be reported separately from the score table.

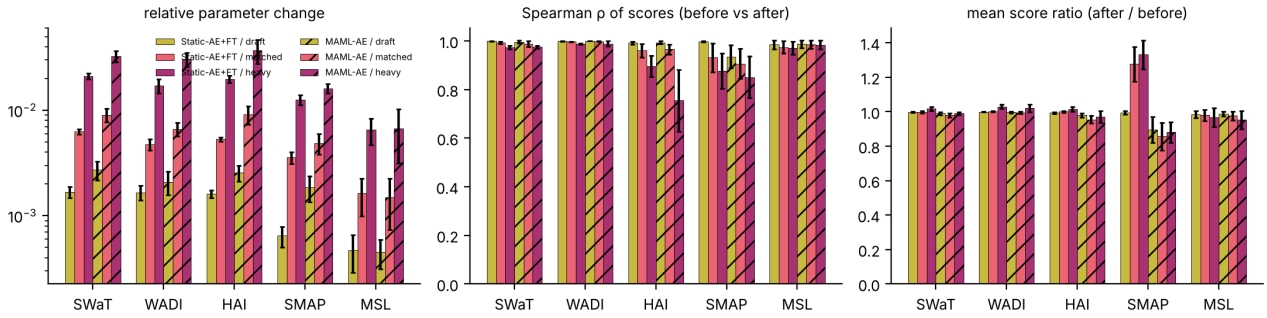


Figure 4. The three adaptation instruments per dataset and setting, for the fine-tuned static model (plain bars) and the meta-learned model (hatched). The parameter-change axis is logarithmic.

The pooled ΔAUROC of +0.01 to +0.016 for the meta-model comes almost entirely from SMAP. There, adaptation at the matched setting lifts the meta-model from 0.567 to 0.597 (173 of 320 cells, $p < 0.0001$), and it does so because the un-adapted meta-model is the weakest neural detector on that dataset; the adapted model ends level with the static model ($p = 0.29$) and 0.014 below the magnitude reference. On the other four datasets the adapted and un-adapted meta-models differ by 0.0001–0.0035. The pattern is the one Raghu et al. [8] reported for classification, only sharper. The inner loop that gives the method its name changes the ranking of windows by a few per cent and the AUROC by a few thousandths. For practical purposes the detector being evaluated is the meta-initialisation.

The broken-loop model and the Scratch model show what adaptation does to a network that has never been trained. At the matched setting their parameters move by 0.3–0.4 % and their scores keep $\rho = 1.000$ with their own un-adapted scores, which are the magnitude reference; at the heavy setting they move by 2.5–4 %, more than any trained model, and still keep $\rho \geq 0.96$. Five to fifty gradient steps on at most a hundred windows do not make a detector out of a random network, and they do not make a different detector out of a trained one.

5.4 The deployed threshold costs more than any modelling choice

Table 6 sets the point-wise F1 at the label-free threshold beside the oracle best F1 for four detectors. The gap is 0.19 F1 on SWaT, 0.06–0.10 on WADI, 0.22–0.25 on HAI, 0.15–0.16 on SMAP and 0.12–0.13 on MSL, and on every dataset but WADI it dwarfs every difference between methods in Table 3. On SWaT the static autoencoder reaches F1 0.56 with the threshold calibrated after 20 or 100 support windows but 0.39–0.40 when calibrated after 1 or 5, because the 300-window calibration stretch then starts a few windows earlier and happens to include a short stretch of quieter operation; nothing about the model changed. On WADI every autoencoder alarms on about 60 % of the evaluation stream against an anomaly rate of 6 %, because the calibration stretch at the start of the attack recording is not representative of the rest of it, a drift inside the attack recording itself; the threshold, not the ranking, is why its F1 is 0.11. On HAI, SMAP and MSL the deployed F1 is 0.14–0.22 against oracle values of 0.30–0.46.

Table 6. Deployed point-wise F1 at the peaks-over-threshold level against the oracle best F1, K = 20, mean over seeds. Alarm rate is the share of evaluation windows above the threshold; the anomaly rate is the share of labelled windows.

Dataset	method	F1 (deployed)	best F1 (oracle)	precision	recall	alarm rate	anomaly rate
SWaT	Static-AE	0.563	0.755	0.465	0.759	0.218	0.120
SWaT	MAML-AE (matched)	0.572	0.755	0.464	0.753	0.198	0.120
SWaT	Magnitude	0.407	0.755	0.271	0.816	0.361	0.120
SWaT	Isolation Forest	0.420	0.746	0.294	0.743	0.307	0.120
WADI	Static-AE	0.113	0.176	0.062	0.615	0.590	0.060
WADI	MAML-AE (matched)	0.116	0.212	0.064	0.653	0.611	0.060
WADI	Magnitude	0.127	0.325	0.069	0.774	0.669	0.060
WADI	Isolation Forest	0.226	0.320	0.142	0.552	0.232	0.060
HAI	Static-AE	0.203	0.456	0.360	0.327	0.113	0.021
HAI	MAML-AE (matched)	0.220	0.437	0.389	0.312	0.073	0.021
HAI	Magnitude	0.207	0.432	0.375	0.322	0.067	0.021
HAI	Isolation Forest	0.058	0.109	0.059	0.109	0.038	0.021
SMAP	Static-AE	0.154	0.305	0.176	0.190	0.110	0.132
SMAP	MAML-AE (matched)	0.142	0.303	0.164	0.190	0.104	0.132
SMAP	Isolation Forest	0.019	0.225	0.127	0.037	0.019	0.132
MSL	Static-AE	0.192	0.324	0.266	0.220	0.132	0.143
MSL	MAML-AE (matched)	0.204	0.329	0.278	0.225	0.134	0.143
MSL	Magnitude	0.216	0.346	0.297	0.267	0.141	0.143
MSL	Isolation Forest	0.115	0.290	0.228	0.135	0.058	0.143

The oracle column is the number most papers report. Read on its own it says the autoencoders are good detectors on SWaT (0.755) and usable on HAI (0.44–0.46). Read beside the deployed column it says that an operator who set the threshold without labels would see 0.56 and 0.20, and on WADI would switch the detector off within an hour. The ranking of methods is also not the same in the two columns: Isolation Forest has the best deployed F1 on WADI by a wide margin (0.226, because it alarms on 23 % of the stream rather than 60 %) while its oracle F1 is level with the magnitude reference.

5.5 Q3, across plants: transfer reaches the target-trained model only where the target is easy, and meta-training adds nothing

Table 7 gives the six transfer directions at K = 20 and K = 100 (Figure 5 maps the K = 20 values). The target decides everything. With SWaT as target every model in the shared 32-component space, including the random network, is between 0.824 and 0.838, and the model trained on SWaT itself is at 0.835: in that space SWaT's attacks are detected by magnitude, as they were in the native space, and there is nothing for transfer to transfer. With WADI as target every model, including the one trained on WADI, is between 0.47 and 0.55: the 32 components keep 91 % of WADI's normal-operation variance but not the part of it that separates attacks, and no source helps. With HAI as target the two source-trained models reach 0.87 from either SWaT or WADI, against 0.895 for the target-trained model and 0.78 for the random network; this is the one direction in which pre-training on another plant carries over something that K target windows cannot supply.

Table 7. Cross-plant transfer, AUROC on the target's attack recording(s), mean ± sd over five seeds, matched adaptation. Source-static is the static autoencoder trained on the source plant and adapted on K target windows; MAML-transfer is the meta-model trained on the source plant's regimes and adapted the same way.

method / K	HAI → SWaT	HAI → WADI	SWaT → HAI	SWaT → WADI	WADI → HAI	WADI → SWaT
Target-static	0.835 ± 0.001	0.540 ± 0.018	0.895 ± 0.006	0.540 ± 0.018	0.895 ± 0.006	0.835 ± 0.001
Source-static, no adaptation	0.833 ± 0.002	0.512 ± 0.052	0.868 ± 0.005	0.550 ± 0.019	0.864 ± 0.005	0.838 ± 0.000
Source-static, adapted / 20	0.832 ± 0.002	0.523 ± 0.046	0.874 ± 0.004	0.545 ± 0.019	0.870 ± 0.006	0.837 ± 0.000
Source-static, adapted / 100	0.831 ± 0.002	0.522 ± 0.047	0.874 ± 0.006	0.542 ± 0.019	0.869 ± 0.005	0.837 ± 0.000
MAML-transfer, no adaptation	0.832 ± 0.001	0.472 ± 0.012	0.870 ± 0.000	0.491 ± 0.029	0.862 ± 0.010	0.837 ± 0.001
MAML-transfer / 20	0.829 ± 0.000	0.510 ± 0.009	0.875 ± 0.001	0.499 ± 0.024	0.874 ± 0.005	0.836 ± 0.001
MAML-transfer / 100	0.828 ± 0.001	0.506 ± 0.009	0.875 ± 0.001	0.495 ± 0.023	0.873 ± 0.004	0.835 ± 0.001
Scratch / 20	0.824 ± 0.002	0.490 ± 0.010	0.780 ± 0.004	0.490 ± 0.010	0.780 ± 0.004	0.824 ± 0.002

Meta-training on the source plant's regimes does not change the transfer. Paired on cells, the meta-model and the static source model adapted the same way differ by +0.0004 (SWaT → HAI, p = 0.50), +0.0034 (WADI → HAI, p = 0.11), −0.003 (HAI → SWaT), −0.001 (WADI → SWaT), −0.015 (HAI → WADI, p = 0.56) and −0.046 (SWaT → WADI, 0 of 10 cells); the heavy setting does not change the picture. The adaptation step itself is worth +0.005 to +0.037 to the meta-model on the HAI and WADI targets, which again says that its initialisation starts below the static model and adaptation brings it level, not above.

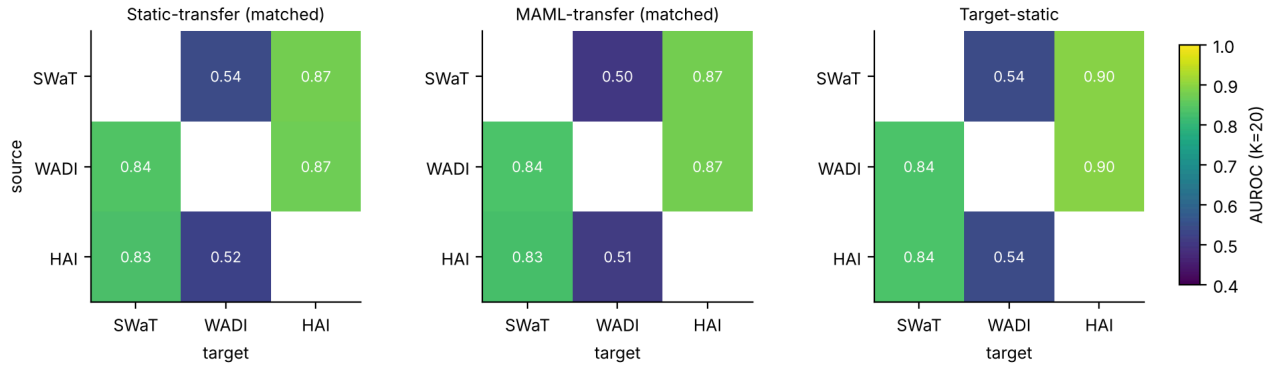


Figure 5. Cross-plant transfer at K = 20: AUROC on the target plant for the adapted static source model, the adapted meta-model and the model trained on the target.

5.6 Task diversity does not predict a meta-learning gain

The explanation we offered in our own earlier draft for a weak meta-learning result was that a single plant does not supply enough distinct tasks. The per-stream diversity scores let that be tested. The silhouette of the regime clustering is 0.18 on WADI, 0.23 on HAI, 0.31 on SWaT, 0.39 on average over MSL channels and 0.66 on average over SMAP channels; the gain of the matched meta-model over the static model on the same streams is +0.008, −0.056, −0.009, −0.003 and +0.005. Across the 120 (channel, seed) cells of the two telemetry sets, where the diversity score and the gain are both per channel, the Spearman correlation is −0.20 ($p = 0.029$): if anything, the more distinct the regimes in a channel, the less the meta-learned model gains (Figure 6). These data do not support the hypothesis, and the weak effect they do show runs the other way.

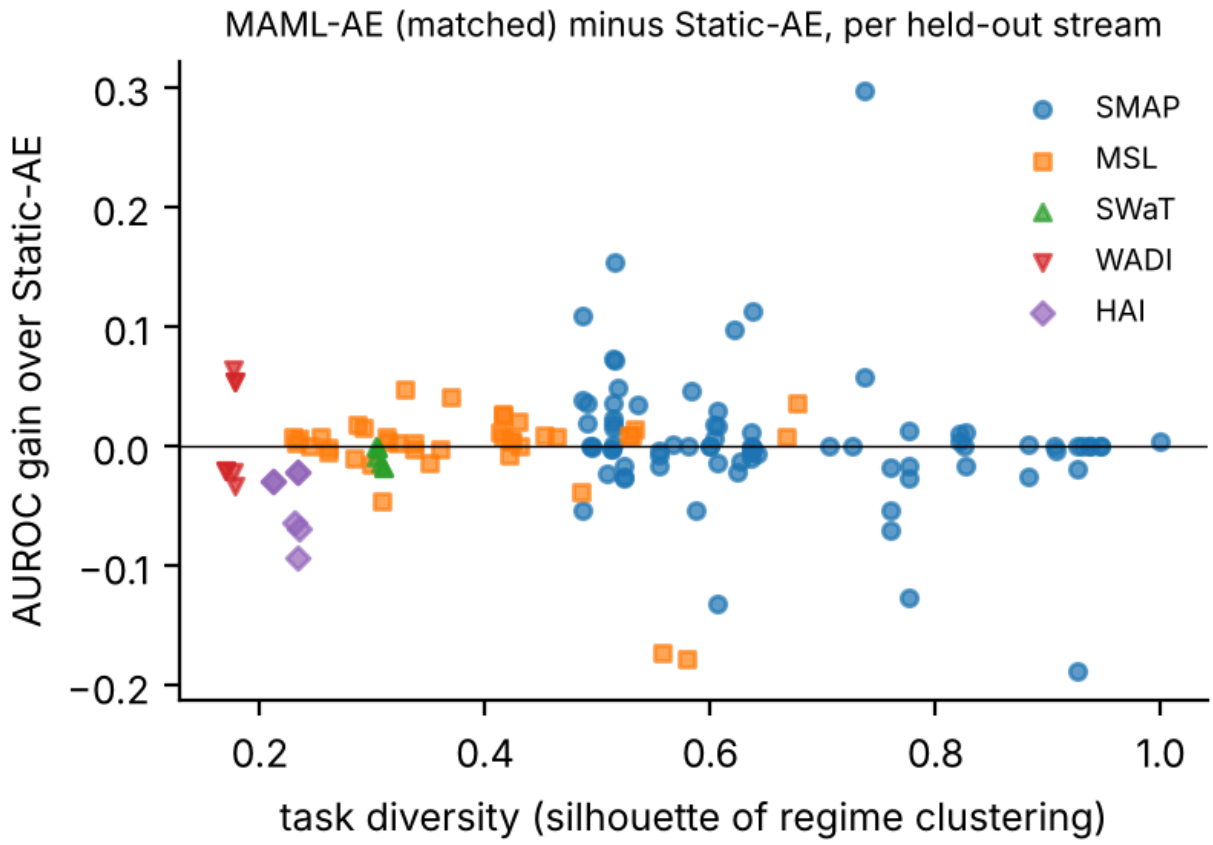


Figure 6. Gain of the matched meta-model over the static model per held-out stream against the stream's task-diversity score. SMAP and MSL points are channels; the testbeds contribute one point per seed.

5.7 Longer meta-training

Table 2 left open whether 600 outer steps had simply stopped the meta-learner short. We therefore repeated the three testbeds with 3,000 outer steps for seeds 0 and 1, everything else unchanged, and compared with the same seeds of the main run. Five times the meta-training lowered the best post-adaptation validation loss from 0.46 to 0.33 on SWaT, 0.71 to 0.58 on WADI and 0.60 to 0.42 on HAI, so the meta-model was indeed still improving as a reconstructor; it remained 1.3 (HAI) to 2.7 (SWaT, WADI) times the static model's validation loss. Detection did not follow. The AUROC of the matched meta-model moved from 0.855 to 0.862 on SWaT, 0.597 to 0.593 on WADI and 0.873 to 0.864 on HAI, against 0.865, 0.581 and 0.898 for the static model on the same seeds; the un-adapted meta-model moved by 0.002 to 0.007. The adaptation instruments were unchanged to the second decimal (relative parameter change 0.005–0.007 at the matched setting, rank correlation 0.90–1.00). Longer meta-training gives a meta-model that reconstructs better and detects the same.

6 Discussion

6.1 What the three questions came to

The meta-learner trains (Q1), and the only quantity that shows it is the one almost nobody reports. The broken loop is not an exotic mistake: it is the natural way to write first-order MAML on top of a framework whose optimisers act on parameter objects, and it produces a run whose loss curve, validation loss and final score table all look like a result. The 25 control runs give its signature, a parameter update of exactly zero at

every outer step, and that signature costs one line of logging. We think a paper that reports a MAML-trained detector should print the norm of the meta-update, or the distance between the final meta-parameters and the initialisation, in the same table as its scores.

K-shot adaptation does almost nothing to this detector (Q2). At the settings that the literature uses and that we used ourselves before this study, it moves the autoencoder by a tenth to half of one per cent of its parameter norm and leaves the ranking of windows with a rank correlation above 0.93. A fifty-step adaptation at ten times the learning rate moves it by two per cent and reorders a little more, and still changes AUROC by a few thousandths. The one dataset on which adaptation produced a measurable gain, SMAP, is the one on which the un-adapted meta-model was the worst neural detector; adaptation repaired a poor initialisation to the level of the static model, which is a different claim from the one usually made. The reason is plain enough. A recurrent autoencoder with 70–100 thousand parameters, a clipped gradient and a handful of normal windows has nowhere to go in five or fifty steps: the support loss on those windows falls, as it did in every run, but the function the network computes on the rest of the stream is the function it computed before. The practical promise of meta-learning is that a deployed detector can be pulled into a new regime with a few windows. For this class of model that promise is not kept. The fix is not a better meta-initialisation but a different adaptation mechanism, one that updates the threshold and the normalisation statistics, since those are the parts of the detector that a few windows can inform.

On detection itself (Q3), the meta-learned model never beat the static autoencoder by more than 0.01 AUROC within a plant and was worse by 0.056 on HAI; across plants it matched the statically pre-trained source model in every direction and never exceeded it. These are the findings of our earlier draft, now with five seeds, paired tests, five benchmarks and the two controls in place, and we stand by them. The instruments added the part we did not expect: on SWaT, WADI, SMAP and MSL the squared magnitude of the standardised window, with no parameters at all, is level with or better than every autoencoder, trained or meta-trained, and only HAI shows a clear benefit from training (and there the flat autoencoder beats the recurrent one). The random network is the magnitude detector in disguise, which is why the broken loop’s “meta-model” scored 0.858 on SWaT. Wu and Keogh [20] argued that the standard benchmarks are solved by trivial detectors; our numbers are a concrete instance, with the trivial detector named and the margin measured, on the two testbeds that industrial meta-learning papers most often use.

6.2 Why the task-diversity explanation did not hold

It is tempting, when a meta-learner fails to help, to say that the tasks were too alike for meta-learning to have anything to learn across. We said it ourselves. The per-channel diversity scores over 120 telemetry cells do not support it: the correlation between the silhouette of a channel’s regimes and the meta-model’s gain over the static model is weakly negative. A more likely account is in Section 5.1: the meta-objective is the post-adaptation reconstruction loss on a held-out regime, and after 600 outer steps that loss is still far above what plain training on the pooled windows reaches, so the meta-model is a worse reconstructor that adaptation cannot repair in five steps. Section 5.7 showed that five times the meta-training narrows that gap without changing the detector, which points at the detector’s dependence on window magnitude (Section 5.2) rather than at the amount of training. Either way, a diversity argument should come with a diversity measurement and a sample size. With both in hand, in this case it fails.

6.3 What to report when reporting a meta-learned detector

The study points to five things a reader needs in order to judge whether an adaptation did anything, and none of them costs more than a few lines of code.

1. The norm of the meta-parameter update per outer step, or the distance between the final meta-parameters and the initialisation, so that a silent gradient-path fault is visible.
2. The relative parameter change and the rank correlation of anomaly scores before and after K-shot adaptation, at the adaptation setting actually used.
3. The un-adapted meta-model and a random initialisation adapted the same way, as rows of the main table.
4. A parameter-free reference, such as the window magnitude, so that the share of a score that comes from the data rather than the model is known.
5. The F1 at a threshold set without labels beside the oracle best F1, with the alarm rate, so that a reader can see what a deployment would have seen.

Together with seeds and paired tests, which Henderson et al. [26] and Pineau et al. [27] have asked for across machine learning, these would have changed the conclusions of our own earlier draft and, we suspect, of some published ones.

6.4 Limitations

The detector is one architecture at one size, and the meta-learner is one first-order variant with fixed inner learning rates. MAML++ or Meta-SGD learn those rates and might adapt more, though they would meet the same ceiling from the magnitude reference. Meta-training was limited to 600 outer steps in the main study and 3,000 in Section 5.7, on CPU; the five-fold longer run changed the validation loss and not the detector, but a still longer run or a larger meta-batch has not been tried. The regime tasks come from one clustering method with one window summary; a different task construction could give the meta-learner different things to learn across, which is exactly the experiment the diversity instrument makes possible. The transfer bridge is a linear projection fitted on normal data, and the WADI result in particular shows that the bridge can destroy what the target's attacks look like; a learned bridge might behave differently. The two testbeds contribute one attack recording each, so their numbers rest on five seeds of model training over the same stream, and their paired tests have 20 cells; the telemetry sets and HAI carry the statistical weight. Everything here is offline: the model is adapted once and scored on a fixed stream. Whether a detector that adapts again and again as a stream drifts needs meta-learning is a separate question with its own protocol. Nothing in this paper settles it.

7 Conclusion

We asked whether a meta-learned LSTM autoencoder for few-shot anomaly detection trains, adapts and detects, and we measured each rather than inferring it from a score table. It trains, once a one-line gradient-path fault is fixed, and a control that leaves the fault in shows that the fault is invisible to every quantity usually reported except the size of the meta-update. It barely adapts: K-shot adaptation at the settings in use moves the network by less than one per cent and leaves the anomaly ranking essentially unchanged on all five benchmarks, with one dataset on which adaptation repairs a poor initialisation to the level of a static model. And it does not detect better than a static autoencoder within a plant or across plants, while on four of five benchmarks neither of them detects better than the squared magnitude of the standardised window. The deployed threshold, set without labels, costs 0.06–0.25 F1 against the oracle, which on four of the five benchmarks is more than any modelling choice in the study. We release the loaders, the meta-learner with its control, the runner and every table, and we propose that reports of meta-learned detectors include the five measurements that made these conclusions possible.

Data and code availability

HAI 21.03, SMAP and MSL are public. SWaT and WADI are available from iTrust on request under their data-use agreement and are not part of our release. The code (loaders, the LSTM autoencoder, the first-order MAML implementation with the broken-pattern control, the regime construction, the runner and the table and figure scripts) and every result table are available at the repository named in the submission.

Declaration

The authors have no competing interests to declare. The work received no external funding.

References

[1] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Proceedings of the 34th international conference on machine learning*, in PMLR, vol. 70. 2017, pp. 1126–1135.

[2] J. Moon, Y. Noh, S. Jung, J. Lee, and E. Hwang, “Anomaly detection using a model-agnostic meta-learning-based variational auto-encoder for facility management,” *Journal of Building Engineering*, vol. 68, p. 106099, 2023, doi: 10.1016/j.jobe.2023.106099.

[3] Y. Yan, Y. Yang, F. Shen, M. Gao, and Y. Gu, “Meta learning-based few-shot intrusion detection for 5G-enabled industrial internet,” *Complex & Intelligent Systems*, vol. 10, no. 3, pp. 4589–4608, 2024, doi: 10.1007/s40747-024-01388-1.

[4] T. Feng, Q. Qi, L. Guo, and J. Wang, “Meta-UAD: A meta-learning scheme for user-level network traffic anomaly detection.” 2024.

[5] J. Tian, M. Li, Z. Wang, L. Chen, J. Qin, and R. Zhang, “OMLog: Online log anomaly detection for evolving system with meta-learning.” 2024.

[6] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang, “An empirical study on TensorFlow program bugs,” in *Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis (ISSTA)*, 2018, pp. 129–140. doi: 10.1145/3213846.3213866.

[7] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, “Taxonomy of real faults in deep learning systems,” in *Proceedings of the ACM/IEEE 42nd international conference on software engineering (ICSE)*, 2020, pp. 1110–1121. doi: 10.1145/3377811.3380395.

[8] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, “Rapid learning or feature reuse? Towards understanding the effectiveness of MAML,” in *8th international conference on learning representations (ICLR)*, 2020.

[9] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based encoder-decoder for multi-sensor anomaly detection,” *arXiv preprint arXiv:1607.00148*, 2016.

[10] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms.” 2018.

[11] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*, 2014, pp. 4–11. doi: 10.1145/2689746.2689747.

[12] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust anomaly detection for multivariate time series through stochastic recurrent neural network,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2828–2837. doi: 10.1145/3292500.3330672.

[13] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, “USAD: Unsupervised anomaly detection on multivariate time series,” in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 3395–3404. doi: 10.1145/3394486.3403392.

[14] A. Deng and B. Hooi, “Graph neural network-based anomaly detection in multivariate time series,” in *Proceedings of the AAAI conference on artificial intelligence*, 2021, pp. 4027–4035. doi: 10.1609/aaai.v35i5.16523.

[15] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, “Deep learning for time series anomaly detection: A survey,” *ACM Computing Surveys*, vol. 57, no. 1, pp. 15:1–15:42, 2024, doi: 10.1145/3691338.

[16] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, “A review on outlier/anomaly detection in time series data,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 56:1–56:33, 2021, doi: 10.1145/3444690.

[17] L. Ruff *et al.*, “A unifying review of deep and shallow anomaly detection,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 756–795, 2021, doi: 10.1109/JPROC.2021.3052449.

[18] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, “Towards a rigorous evaluation of time-series anomaly detection,” in *Proceedings of the AAAI conference on artificial intelligence*, 2022, pp. 7194–7201. doi: 10.1609/aaai.v36i7.20680.

[19] A. Garg, W. Zhang, J. Samaran, R. Savitha, and C.-S. Foo, “An evaluation of anomaly detection and diagnosis in multivariate time series,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 6, pp. 2508–2517, 2022, doi: 10.1109/TNNLS.2021.3105827.

[20] R. Wu and E. J. Keogh, “Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2421–2429, 2023, doi: 10.1109/TKDE.2021.3112126.

[21] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, “Anomaly detection in streams with extreme value theory,” in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 1067–1075.

[22] A. Antoniou, H. Edwards, and A. Storkey, “How to train your MAML,” in *7th international conference on learning representations (ICLR)*, 2019.

[23] Z. Li, F. Zhou, F. Chen, and H. Li, “Meta-SGD: Learning to learn quickly for few-shot learning.” 2017.

[24] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, “Meta-learning in neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5149–5169, 2022.

[25] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning,” *ACM Computing Surveys*, vol. 53, no. 3, pp. 63:1–63:34, 2020, doi: 10.1145/3386252.

[26] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proceedings of the AAAI conference on artificial intelligence*, 2018, pp. 3207–3214. doi: 10.1609/aaai.v32i1.11694.

[27] J. Pineau *et al.*, “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[28] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[29] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[30] R. Kermenov, G. Nabissi, S. Longhi, and A. Bonci, “Anomaly detection and concept drift adaptation for dynamic systems: A general method with practical implementation using an industrial collaborative robot,” *Sensors*, vol. 23, no. 6, p. 3260, 2023, doi: 10.3390/s23063260.

[31] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[32] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *2008 eighth IEEE international conference on data mining*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[33] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd international conference on learning representations (ICLR)*, 2015.

[34] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Critical information infrastructures security (CRITIS 2016)*, in Lecture notes in computer science, vol. 10242. Springer, 2017, pp. 88–99.

[35] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, “WADI: A water distribution testbed for research in the design of secure cyber physical systems,” in *Proceedings of the 3rd international workshop on cyber-physical systems for smart water networks*, 2017, pp. 25–28.

[36] H.-K. Shin, W. Lee, J.-H. Yun, and B.-G. Min, “Two ICS security datasets and anomaly detection contest on the HIL-based augmented ICS testbed,” in *Cyber security experimentation and test workshop (CSET ’21)*, 2021, pp. 36–40.

[37] H.-K. Shin, W. Lee, J.-H. Yun, and H. Kim, “HAI 1.0: HIL-based augmented ICS security dataset,” in *13th USENIX workshop on cyber security experimentation and test (CSET 20)*, 2020.

[38] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, “Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding,” in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 387–395. doi: 10.1145/3219819.3219845.

[39] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, p. 20150202, 2016, doi: 10.1098/rsta.2015.0202.